

Advanced techniques in applied economics

Lecture 5: Latent variable models — from trees to neural nets

Piotr Zwiernik



**Universitat
Pompeu Fabra**
Barcelona

Statistics, Probability
and Machine Learning
Research Group



Barcelona School of Economics

Spring 2026

Latent structure as hidden economic mechanism

What changes today?

In the previous lectures, graphs described dependence among *observed* variables. Today we allow some important variables to be **hidden**.

Why hidden variables?

- market-wide shocks are not directly observed,
- consumers or firms belong to hidden segments,
- hierarchical structures may be only partially measured.

Dimension reduction

PCA and related models assume low-dim latent structure in the data.

Main question

Can a small number of latent variables explain complicated patterns in the data?

Main message: latent variables can turn a complicated observed dependence pattern into a simple hidden structure.

Roadmap for today

1. Factor analysis and hidden common factors
2. Gaussian mixtures and latent classes
3. Latent tree models and hierarchical structure
4. Message passing / belief propagation intuition
5. Restricted Boltzmann machines as a bridge to neural nets

When dependence is too complicated at the surface, look for a simpler hidden layer underneath.

Part 1: Factor analysis

Why economists care about factor models

Finance

A large collection of returns often moves because of a few common factors:

- market return,
- sectoral shocks,
- liquidity or volatility factors.

Applied micro and macro

Latent variables can represent

- ability,
- sentiment,
- business-cycle conditions,
- hidden household or firm types.

A factor model says: many variables may comove because they are driven by the same hidden source.

The factor analysis model

The basic Gaussian factor model is

$$X = \mu + WZ + \varepsilon, \quad Z \sim N_r(0, I_r), \quad \varepsilon \sim N_m(0, \Psi), \quad Z \perp\!\!\!\perp \varepsilon,$$

where Ψ is diagonal.

Interpretation

- $Z \in \mathbb{R}^r$ are the latent factors,
- $W \in \mathbb{R}^{m \times r}$ is the loading matrix,
- Ψ captures variable-specific noise not explained by the common factors.

Covariance form

The induced covariance matrix is

$$\Sigma = WW^\top + \Psi.$$

So dependence is split into a low-rank common part and a diagonal idiosyncratic part.

A one-factor economic example

Suppose m observed returns satisfy

$$X_i = \lambda_i F + \varepsilon_i, \quad i = 1, \dots, m,$$

where F is a latent factor, and all ε_i independent and independent of F .

Covariance implication

For $i \neq j$,

$$\text{cov}(X_i, X_j) = \lambda_i \lambda_j \text{var}(F).$$

Also,

$$\text{cov}(X_i, F) = \lambda_i \text{var}(F),$$

so

$$\text{cov}(X_i, X_j) = \frac{\text{cov}(X_i, F) \text{cov}(X_j, F)}{\text{var}(F)}.$$

Economic reading

The three assets comove because they all load on the same hidden market condition F .

Correlation implication

Let $\rho_i = \text{cor}(X_i, F)$, $\rho_{ij} = \text{cor}(X_i, X_j)$:
 $\rho_{ij} = \rho_i \rho_j$ for all $i \neq j$.

A simple correlation consequence

One-factor correlation pattern

The observed correlation matrix must have the form $\rho_{ij} = \rho_i \rho_j$.

Testable implications

- Taking $1 \leq i < j < k \leq m$ and multiplying the corresponding correlations gives

$$\rho_{ij}\rho_{ik}\rho_{jk} = \rho_i^2 \rho_j^2 \rho_k^2 \geq 0.$$

So under one-factor model, the product of specific three pairwise correlations cannot be negative.

- For any $1 \leq i < j < k < l \leq m$ we have the **tetrad constraints**

$$\rho_{ij}\rho_{kl} = \rho_{ik}\rho_{jl} = \rho_{il}\rho_{jk}.$$

A tiny one-factor check in R

```
R <- matrix(c(1, 0.6, 0.3,  
             0.6, 1, 0.5,  
             0.3, 0.5, 1), 3, 3)
```

```
rho12 <- R[1,2]
```

```
rho13 <- R[1,3]
```

```
rho23 <- R[2,3]
```

```
rho12 * rho13 * rho23
```

```
# implied squared correlations
```

```
rho2 <- c(rho12*rho13/rho23,  
          rho12*rho23/rho13,  
          rho13*rho23/rho12)
```

```
rho2
```

```
# all should lie in [0,1]
```

```
all(rho2 >= 0)
```

```
all(rho2 <= 1)
```

Interpretation

A negative product $\rho_{12}\rho_{13}\rho_{23}$ rules out a one-factor explanation. If the implied values also lie in $[0, 1]$, then a one-factor explanation is at least algebraically possible.

How many factors?

A practical question is: how should we choose the number of factors?

Common heuristics

- scree plot,
- proportion of explained variance,
- information criteria,
- **parallel analysis** (Horn's method).

Horn's parallel analysis

Compare the observed eigenvalues of the sample correlation matrix to the eigenvalues obtained from random data with no factor structure. Retain only those factors whose observed eigenvalues are larger than the random benchmark.

Parallel analysis gives a simple data-driven check against overfitting too many factors.

Identifiability and rotation

The factor model is not uniquely parameterized. If U is an orthogonal matrix, then

$$WZ = (WU)(U^T Z),$$

so the same observed distribution can arise from many different loading matrices.

Why this matters

- the covariance matrix Σ may be identified,
- but the individual factor loadings are not unique,
- so interpretation of the latent factors requires care.

Practical consequence

The statistical model may fit well, but the factors are not automatically interpretable. To get a simpler loading pattern, we often apply a rotation such as varimax.

What does varimax do?

Varimax is an **orthogonal rotation** of the loading matrix.

It rotates the factors to make the loadings more interpretable:

- each variable should load strongly on only a few factors,
- many other loadings should be close to zero.

Psychology example

Suppose we measure many questionnaire items. Before rotation, each item may have moderate loadings on several factors. After varimax, factors may be better aligned with latent personality traits.

Economic analogy

Think of a panel of indicators. After rotation, one factor may look mostly like *business cycle*, another like *financial conditions*, another like *sector-specific demand*.

Varimax does not change the fitted covariance matrix. It only changes the coordinate system used to describe the same latent space.

Choosing and fitting a factor model in R

```
set.seed(1)
library(MASS)
library(psych)

Lambda <- matrix(c(0.8, 0.2,
                  0.7, 0.1,
                  0.6, 0.3,
                  0.1, 0.7,
                  0.2, 0.8), 5, 2,
                byrow = TRUE)
Psi <- diag(c(0.3, 0.4, 0.5, 0.3, 0.4))
Sigma <- Lambda %*% t(Lambda) + Psi
X <- mvrnorm(500, rep(0, 5), Sigma)

fa.parallel(X, fa = "fa")
```

```
fit <- factanal(X, factors = 2,
               rotation = "varimax")
fit$loadings

cor(X)
```

Interpretation

Parallel analysis helps decide how many factors to retain. Varimax then rotates the fitted loading matrix into a simpler and more interpretable form.

Application: Factor Analysis on Personality Traits Data

We analyze real survey data from the `bfi` dataset in the `psych` R package.

The dataset consists of 2,800 responses to 25 personality-related questions.

These questions measure the Big Five Personality Traits:

- Neuroticism (N)
- Extraversion (E)
- Conscientiousness (C)
- Agreeableness (A)
- Openness (O)

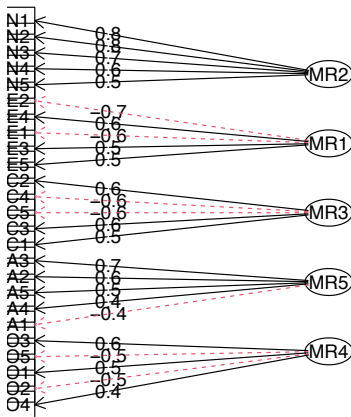
Responses are on the 1-6 scale indicating agreement strength.

We discard demographic variables (gender, age, education).

Factor Analysis Results

PA suggests retaining **five** factors, matching the Big Five personality traits (nice!).

Factor loadings after **varimax rotation** confirm that the extracted factors correspond to the expected latent traits.



What factor analysis does well — and what it misses

Strengths

- compresses many variables into a few hidden factors,
- often fits financial or psychometric data well,
- gives an interpretable covariance decomposition.

Limitations

- loadings are not uniquely identified,
- factors are often global rather than local,
- the model is covariance-based and may miss multimodality.

Factor analysis is good for hidden common shocks, but not for hidden market segments or branching hierarchies.

Part 2: Gaussian mixtures and latent classes

Gaussian mixtures as latent class models

A Gaussian mixture model has density

$$p(x) = \sum_{k=1}^K \pi_k \phi(x; \mu_k, \Sigma_k), \quad \pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$

Latent class interpretation

Introduce an unobserved class label $Z \in \{1, \dots, K\}$ with

$$\mathbb{P}(Z = k) = \pi_k.$$

Conditional on $Z = k$, the data come from a Gaussian distribution with parameters (μ_k, Σ_k) .

Economic reading

A population may consist of hidden segments: conservative vs aggressive investors, low- vs high-productivity firms, or distinct consumer types.

Factor models vs mixture models

Factor model

A continuous latent variable explains dependence through common shocks. Observed clouds are typically elongated.

Mixture model

A discrete latent variable explains dependence through hidden subpopulations. Observed clouds may be multimodal or clustered.

Continuous latent variables explain *common variation*; discrete latent variables explain *hidden segmentation*.

A Gaussian mixture in R

```
set.seed(1)
library(mclust)

x1 <- cbind(rnorm(150, -2, 0.8),
            rnorm(150, 0, 0.6))
x2 <- cbind(rnorm(150, 2, 0.8),
            rnorm(150, 1, 0.6))
X <- rbind(x1, x2)
```

```
fit <- Mclust(X, G = 2)
summary(fit)
plot(fit, what = "classification")
```

Interpretation

The hidden label tells us which latent segment generated each observation. This is a very different explanation of dependence from a factor model.

A quick word on the EM algorithm

Both factor analysis and Gaussian mixtures are often fitted with versions of the EM algorithm.

General latent-variable idea

- **E-step:** estimate the hidden variables given the current parameters,
- **M-step:** update the parameters given the current estimate of the hidden variables.

Examples

- in mixtures, the hidden variable is the class label,
- in factor models, the hidden variable is the latent factor vector.

Part 3: Latent tree models

Why latent tree models?

Sometimes one hidden factor is too crude, but a fully connected latent structure is too complicated.

Latent tree idea

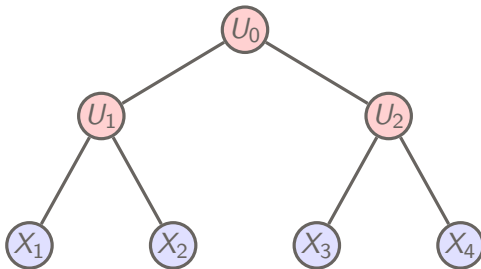
Hidden variables are arranged in a tree, and observed variables appear at the leaves.

Economic reading

This is natural for hierarchical structures such as sectors, sub-sectors, and firms, or aggregate demand decomposed into regional and local components.

Latent tree models sit between one global factor and a fully general latent network.

A simple latent tree picture



Interpretation

X_1 and X_2 are similar because they share a local hidden parent U_1 . At a higher level, both groups are connected through the root U_0 .

Why trees are computationally attractive

Trees are special because many inference problems become simple.

Message passing intuition

If the graph has no loops, information can be sent inward from the leaves and outward from the root. Each local update summarizes everything happening below that node.

Why this matters

Belief propagation is exact on trees. So marginal distributions and conditional probabilities can be computed efficiently even in large latent trees.

Trees are beautiful because global inference reduces to local calculations.

Belief propagation in one sentence

Suppose a tree is rooted at some latent variable.

Upward pass

Each child sends a summary message to its parent. This message tells the parent how likely different hidden states are, given the evidence below.

Downward pass

The parent sends back a message combining information from the rest of the tree. This gives local posterior probabilities at every node.

Intuition

Every message is a compressed summary of a whole subtree.

A tiny hidden Markov / tree-flavored example in R

Even if we do not fit a full latent tree, the computational idea is similar: local hidden states are updated from local evidence.

```
# simple hidden Markov example
library(HMM)
A <- matrix(c(0.9,0.1,
              0.2,0.8), 2, 2,
            byrow = TRUE)
B <- matrix(c(0.8,0.2,
              0.3,0.7), 2, 2,
            byrow = TRUE)
```

```
hmm <- initHMM(c("low","high"),
               c("bad","good"),
               transProbs = A,
               emissionProbs = B)
forward(hmm, c("good","good",
               "bad","good"))
```

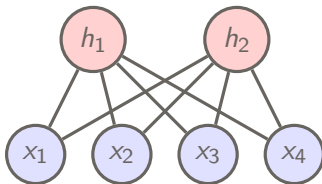
Interpretation

The same message-passing idea underlies more general tree-structured latent models.

Part 4: Restricted Boltzmann machines

Restricted Boltzmann machines

A restricted Boltzmann machine (RBM) is a bipartite latent-variable model with visible units x and hidden units h .



Restriction

There are no visible-visible edges and no hidden-hidden edges. This makes conditional distributions very simple.

Why RBMs matter here

RBMs are still latent-variable models, but they point toward neural networks.

Modeling idea

The hidden units capture a distributed latent representation of the visible variables. Unlike a classical one-factor model, many hidden units can be active at once.

Classical latent models

- one or a few interpretable latent variables,
- emphasis on structure and interpretation.

RBMs / deep learning bridge

- many hidden features,
- emphasis on representation rather than interpretation.

From RBMs to sigmoid activation

For a binary RBM, the joint distribution has the form

$$P(x, h) \propto \exp\left(a^\top x + b^\top h + x^\top W h\right).$$

Because the graph is bipartite,

$$P(h_j = 1 \mid x) = \frac{\exp(b_j + w_j^\top x)}{1 + \exp(b_j + w_j^\top x)} = \sigma(b_j + w_j^\top x),$$

where $\sigma(t) = 1/(1 + e^{-t})$ is the sigmoid function.

Bridge to neural nets

A hidden unit in an RBM activates according to a sigmoid of a linear score. This is one reason RBMs historically formed a bridge from latent-variable models to neural-network representations.

A tiny RBM-style code illustration

In modern practice, one usually works with autoencoders or neural nets rather than classical RBM training, but the basic hidden-layer idea is similar.

```
set.seed(1)
X <- matrix(rbinom(500*4, 1, 0.5),
            500, 4)
W <- matrix(c(1.2,-0.8,
              0.5, 1.1,
              -0.7, 0.9,
              1.0, 0.6), 4, 2,
            byrow = TRUE)
```

```
eta <- X %*% W
Hprob <- 1/(1 + exp(-eta))
head(Hprob)
```

Interpretation

The hidden layer turns observed inputs into latent activation probabilities — the modern representation-learning version of a latent-variable model.

Take-away

- Factor analysis explains dependence through a few hidden common shocks.
- Gaussian mixtures explain dependence through hidden subpopulations.
- Latent tree models explain dependence through hierarchical hidden structure.
- Message passing makes tree-structured inference computationally attractive.
- RBMs show how latent-variable ideas connect naturally to neural-network representations.

Different latent models explain dependence in different ways: common factors, hidden classes, hierarchies, or distributed features.

Looking ahead

Next lecture

We will return to hidden variables from a more causal perspective: unobserved confounding, proxy controls, and what survives when the clean DAG picture breaks.

Bridge forward

Today the latent variables explained dependence. Next time they will also threaten identification.